kiwi captcha

Complete website / edition 4 / 12 October 2026

KiwiCaptcha
The full website.

Every current page, complete documentation, identity system and dated prior website content in a paginated edition.

01 / Self-hosted adaptive anti-abuse

02 / How KiwiCaptcha works

03 / KiwiCaptcha integrations

04 / KiwiCaptcha security and enforcement

05 / KiwiCaptcha privacy model

06 / KiwiCaptcha operator documentation

07 / KiwiCaptcha widget and recovery

08 / KiwiCaptcha visual identity

09 / Useful details in one place.

10 / Current edition and complete prior reference

Live website: https://kiwicaptcha.windyclock5.chatgpt.site
Current source: 272f0ddb888f2ff35446667fcd8ffcb61a09af32
Local interactive examples are design simulations. PDFs show static states; live pages retain the controls.

kiwi captcha

Symfony-first / Self-hosted adaptive anti-abuse

Your application. Your decision.

Computational checks, ownership factors and transaction-bound authorization under your application's policy. First-party runtime, explicit decisions and an auditable enforcement path.

Explore integration → **Try the widget design**

Symfony-first enforcement. Portable verification clients. Your application owns acceptance.

A check, with the decision in view.

Protected action

Submit an application request

Security Check

Complete

Check complete

The host verifies the response kind.

Illustrative component / A completed check supplies a result. The host still verifies the result and enforces its disposition.

why

A check for the request. A policy for the action.

Repeated signup, login, password-reset and protected-request attempts need more than a decorative security badge. Computational work attaches real friction to an attempt; your application verifies the response and decides whether more assurance or a refusal is appropriate.

A person does not solve the PoW by hand. Their device computes it, and an automated client can do the same. KiwiCaptcha's current value combines that friction with self-hosted step-up, quarantine and denial.

Signup

Keep the work and address-ownership carrier tied to the protected scope.

Login

Separate browser work from credential checks and post-credential assurance.

Sensitive actions

Authorize the specific server-owned operation, not a reusable general pass.

Repeated requests

Bound issuance, verification and aggregate pressure around your policy.

Source notes

README.md

docs/cost-to-abuse.md

mechanism

Challenge. Work. Decision.

Your application issues a scoped, expiring challenge. The browser computes the selected work. Your server checks the signed context, consumes the proof under the configured storage rules and applies the final policy.

A completed widget is a response to verify. A verified factor is still subject to demand finalization. Keep both boundaries visible in the product and in your integration.

See the full lifecycle ↗

Source notes

README.md

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/StepUpDemandFinalizer.php

packages/kiwicaptcha-wasm/assets/widget-driver.js

control

The enforcement stays in your application.

Symfony is the current first-class complete enforcement plane. It composes challenge issuance, risk, additional factors, quarantine and denial around the application.

Other language SDKs support local verification and risk clients with explicit capabilities. Browser and native wrappers manage the client lifecycle; they do not become an authorization service.

Compare integration roles ↗

Source notes

README.md

packages/kiwicaptcha-client-core/README.md

packages/kiwicaptcha-verifier/README.md

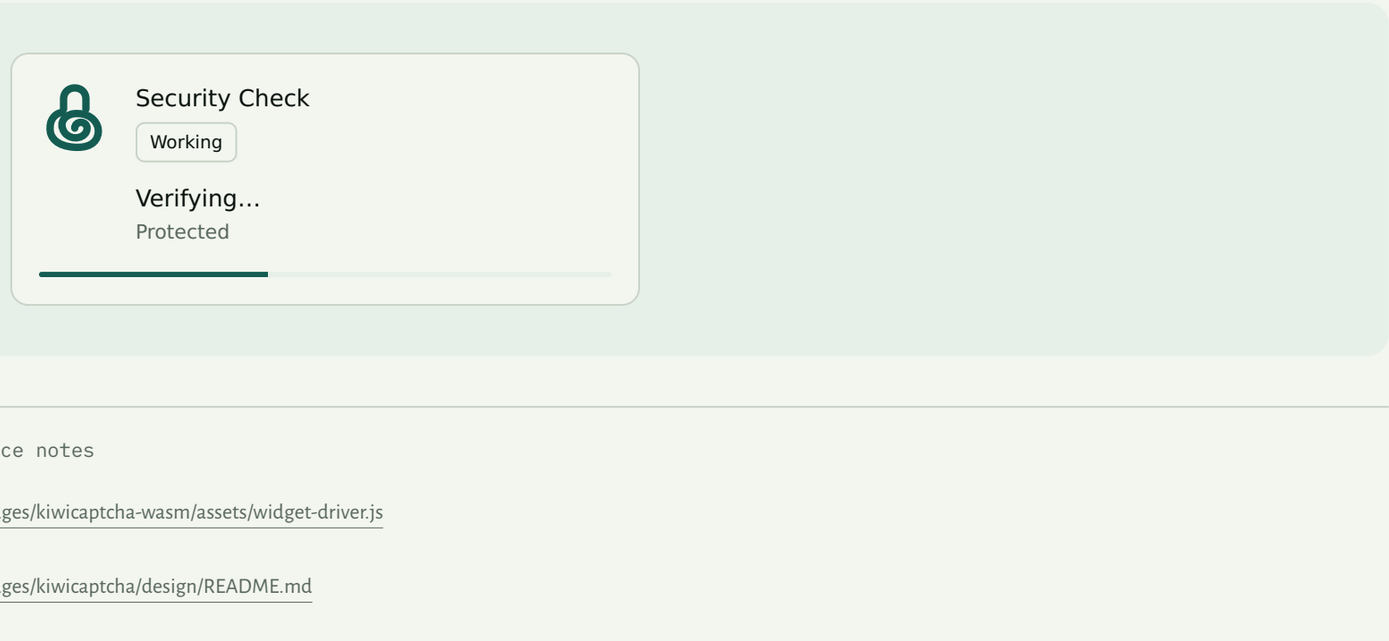
widget

Quiet status. Useful recovery.

The baseline widget shows a static Spiral Lock, readable status and a single decorative work track. It exposes native recovery rather than a checkbox or puzzle.

Current adaptive flows can request address ownership or route a principal to the application's factor flow. Those are typed policy demands, with their own lifecycle and scope.

Inspect the widget states



Source notes

packages/kiwicaptcha-wasm/assets/widget-driver.js

packages/kiwicaptcha/design/README.md

privacy

First-party defaults you can inspect.

The default runtime path stays on your application's origin. Behavioral telemetry is off by default, and strict privacy mode forces its documented restrictions.

Adaptive continuity and optional factors have a specific data contract. Read the configuration and retention scope instead of relying on an absolute no-data slogan.

Read the privacy model ↗

Source notes

packages/kiwicaptcha/integrations/symfony/docs/privacy.md

packages/kiwicaptcha/integrations/symfony/docs/claims-registry.md

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/EmailOwnershipStepUp.php

economics

Friction is useful. It is not the whole policy.

The dedicated cost analysis finds that browser-feasible default work does not price out valuable abuse by itself. Your scope's value and complete enforcement path matter.

Current source provides completable carriers and a doctor that checks their prerequisites. A strong-looking configuration that leaves visitors unable to finish a demanded factor is not a finished deployment.

Understand policy and limits ↗

Source notes

docs/cost-to-abuse.md

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/ProtectionProfileDefaults.php

packages/kiwicaptcha/integrations/symfony/src/Command/KiwiCaptchaDoctorCommand.php

start

A source-grounded path to integration.

Begin with the repository's minimal balanced flow. Configure the secret, canonical origin and Redis-backed services, render the widget, and keep verification server-side.

When you adopt an abuse posture, wire its factors, delivery, routes, operation identity and protocol prerequisites before accepting its doctor result as your deployment gate.

Minimal Symfony configuration

```
kiwi_captcha:
    protection_profile: balanced
    secret_key: '%env(KIWI_SECRET_KEY)%'
    public_base_url: '%env(KIWI_PUBLIC_URL)%'
    redis_dsn: '%env(KIWI_REDIS_DSN)%'
```

Copy example

The repository's verified-flow example. This is the balanced baseline, not a complete abuse_first enforcement deployment.

Server configuration after manual repository installation; environment values must be supplied by the application.

Open the operator guide ↗

Source notes

packages/kiwicaptcha/integrations/symfony/docs/getting-started.md

packages/kiwicaptcha/integrations/symfony/docs/configuration.md

packages/kiwicaptcha/integrations/symfony/src/Command/KiwiCaptchaDoctorCommand.php

---

---

faq

The questions that change a deployment.

Computation, factor completion and authorization are distinct. The current source snapshot and its exact limits are available throughout this site.

Does KiwiCaptcha prove that a visitor is human?

No. Computational work proves a selected computation under its challenge rules. Automated clients can pay that work too. Email ownership establishes control of an address; a principal factor supports the application's configured assurance. None is a universal proof of humanness.

What is the complete first-class integration today?

Symfony is the current complete enforcement plane: issuance, risk decisions, step-up, quarantine and denial. Other language SDKs are verification/risk clients and middleware with their own capability contracts.

Does a completed widget authorize the request?

No. The browser has a response token or, in the ownership path, a grant. The application must verify the response and its lifecycle. A factor must also pass the demand finalizer before an operation or pending login is authorized.

Can raw work price out valuable abuse?

The published default-rung analysis says it cannot at the browser-feasible costs measured. Use a scope-appropriate, completable step-up or denial policy, and bound issuance and verification resources.

Can I install a current release from a registry?

The reviewed README and Symfony guide still state that Rust/Symfony registry publication and the Flex recipe are pending. GitHub's latest listed snapshot is v1.6.31 from August 2026; it is not the current source tree. Use the source integration guides and check current release status before rollout.

Read every answer ↗

Source notes

README.md

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/StepUpDemandFinalizer.php

identity

A product with its own identity.

The Spiral Lock remains the canonical symbol. Fixed humanist lettering, teal and a clear reading rhythm give KiwiCaptcha a complete visual language within the Bel Consulting product family.

Download native artwork and use the independent guide. Historical source analysis remains available with its original date and commit.

Open the brand system ↗

Source notes

packages/kiwicaptcha/resources/kiwi-mark.svg

packages/kiwicaptcha/design/README.md

Download the complete website PDF ↓ All questions and answers →

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

MECHANISM / FROM REQUEST TO DECISION

A proof has a context.

The challenge, its lifecycle and the application's policy travel together. Computational work is one input to acceptance; it never identifies a person by itself.

sequence

Four steps, with a server boundary.

The client handles work and presentation. The application controls scope, storage, verification and acceptance.

01

Issue

Sign and store a challenge with nonce, scope, selected algorithm, validity and policy context.

02

Compute

The client runs the selected computation within the protocol caps and produces a response.

03

Verify

Run the cheap gates first. Consume atomically where the production storage contract supports it, derive once and retain a deterministic result.

04

Enforce

Apply the scope's configured policy: proceed, request more assurance, chain stronger work, quarantine or deny where the complete plane implements it.

Source notes

README.md

packages/kiwicaptcha/integrations/symfony/docs/claims-registry.md

algorithms

Different work. Explicit contracts.

SHA-256, Argon2id and the optional sequential extension have different resource properties. A stronger-sounding algorithm is not a proof of humanity.

Work	Mechanism	Client/server limit
SHA-256	Hash search against leading-zero difficulty	Rust/WASM plus pure-JS fallback. Browser cap and cheap server derivation are documented.
Argon2id	Memory-hard search under signed parameters	Requires the supported WASM/ worker path in the browser. Bound memory and verification concurrency; no silent weak fallback.
RSW extension	Sequential time-lock squaring under modulus identity	Experimental/versioned contract. Browser/native same-hardware speed asymmetry is unmeasured; do not invent a price or speed ratio.

Core algorithm contract ↗

Measured economic limits ↗

Source notes

README.md

packages/kiwicaptcha/README.md

docs/cost-to-abuse.md

cheap / gates

Check the context before spending the proof budget.

The verifier checks structure, authentication, validity, scope, binding and the applicable policy context before expensive proof work. Per-nonce attempt accounting and aggregate controls bound verification cost.

Those mechanisms do not make the application free to leave challenge issuance unbounded. Storage, trusted proxy policy, clock and capacity remain part of the operating contract.

Source notes

README.md

SECURITY.md

packages/kiwicaptcha/integrations/symfony/docs/operations.md

---

single / use

One consume. Deterministic recovery.

Atomic production storage admits one pending-to-consumed transition and proof derivation. The consumed record retains its result, so an exact-operation recovery can reproduce the original outcome.

A different operation cannot treat the same token as a fresh successful request. Ordinary SiteVerify replay and exact idempotency recovery have distinct mappings; a temporary unavailable result is not a guessed success.

Source notes

packages/kiwicaptcha/integrations/symfony/docs/claims-registry.md

packages/kiwicaptcha/integrations/symfony/docs/siteverify.md

packages/kiwicaptcha/integrations/symfony/docs/security-hardening.md

binding

Bind sensitive work to a server-owned operation.

A request binding can correlate a basic anti-abuse flow. Sensitive authorization needs a backend-issued identity and the canonical protected contents.

Current demand finalization checks operation identity and content separately. A proof for transaction A does not authorize transaction B, even if the visible amounts match. The application supplies the authoritative logical operation id, not an unchecked raw client value.

Host attribute

`KiwiCaptchaValidator::OPERATION_ID_ATTRIBUTE`

Current literal

`_kiwi_captcha_operation_id`

Scope contract

`operation_identity_source: request_attribute`

Source notes

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/Configuration.php

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/StepUpDemandFinalizer.php

packages/kiwicaptcha/integrations/symfony/src/Validator/Constraints/KiwiCaptchaValidator.php

factors

An additional factor needs a complete demand.

A current anonymous ownership demand can deliver an address code and return a scope/session-bound grant. A principal-carried demand belongs to the application's authentication flow and can use the enabled reference factors.

A passing factor is not authorization. The shared finalizer checks binding, claims the demand, reassesses policy, settles its terminal state and only then applies journaled effects such as login restoration or completion credit.

Term	Meaning
FactorVerified	The factor assessment passed.
AuthorizedStepUp	The bound demand reached an authorized terminal after its checks.
Protected action	The host proceeds under the authoritative decision and operation identity.

Source notes

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/EmailOwnershipStepUp.php

packages/kiwicaptcha/integrations/symfony/src/Controller/StepUpController.php

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/StepUpDemandFinalizer.php

extensions

Supplementary evidence keeps its limits.

Timing is a heuristic; IP binding mitigates relay; decoys and client telemetry are supplementary. The execution dimension is not browser attestation, and native clients have narrower execution contracts.

Do not turn an optional dimension into a human-proof or browser-proof seal. Operators should inspect the version and rollout contract before arming it.

Source notes

SECURITY.md

packages/kiwicaptcha/README.md

packages/kiwicaptcha/integrations/symfony/docs/configuration.md

next

Choose a complete host contract.

Use the Symfony plane when you need the current complete adaptive enforcement flow. For other stacks, start from the verification client or local sidecar and preserve its stated capability boundary.

Integration roles ↗

Lifecycle and recovery ↗

Operating requirements ↗

Source notes

README.md

packages/kiwicaptcha-verifier/README.md

Download the complete website PDF ↓ All questions and answers →

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

kiwi captcha

INTEGRATIONS / START WITH YOUR STACK

Use the right layer.

The complete enforcement plane, a verification client and a browser wrapper solve different jobs. Choose the contract your application actually needs.

roles

One product. Distinct integration roles.

Symfony is the first-class full enforcement host today. The underlying language-neutral cores and SDKs remain independently useful, with explicit verification and client capabilities.

Layer	Role	Boundary
Symfony	Issuance, risk, factors, quarantine and denial	Current complete first-class enforcement integration.
Rust/PHP cores	Embedding issuance, verification and risk contracts	Host supplies its application integration and operating obligations.

Layer	Role	Boundary
Server SDKs	Local verification, stores, middleware and outcome clients	Not equivalent full enforcement planes.
Browser wrappers	Shared script/render/event lifecycle	Tokens still require authoritative server verification.
Native clients	Off-UI-thread work and client status/token assembly	Backend verifies; execution-armed/ capability limits differ.
Verifier sidecar	Local HTTP/Unix-socket verification for other hosts or delegated dimensions	Co-located service contract; not a third-party authorization service.

Source notes

README.md

packages/kiwicaptcha-client-core/README.md

packages/kiwicaptcha-verifier/README.md

symfony

The complete plane starts in Symfony.

Follow the repository's manual installation guide, register the bundle, and provide the signing secret, canonical public origin and Redis DSN through the environment.

The four-key balanced configuration is a real documented baseline with current smoke fixtures. It does not finish the abuse profile's additional factor and deployment requirements.

Minimal Symfony configuration

```
kiwi_captcha:
    protection_profile: balanced
    secret_key: '%env(KIWI_SECRET_KEY)%'
    public_base_url: '%env(KIWI_PUBLIC_URL)%'
    redis_dsn: '%env(KIWI_REDIS_DSN)%'
```

Copy example

The repository's verified-flow example. This is the balanced baseline, not a complete abuse_first enforcement deployment.

Server configuration after manual repository installation; environment values must be supplied by the application.

Manual bundle registration

```
return [
    // ...
    BelConsulting\KiwiCaptchaBundle\KiwiCaptchaBundle::class => ['all' => true],
];
```

Copy example

From the manual installation guide. This is part of the source-install path while registry publication/Flex recipe remain pending.

Integrating application's config/bundles.php; require the repository bundle and its dependencies first.

Inspect the installed wiring

```
bin/console kiwicaptcha:doctor
```

Copy example

The command inspects the services and configuration actually installed in the application. A FAIL returns a nonzero exit; WARN names something the command could not establish or a deployment decision still open.

Inside the integrating Symfony application. No application deployment or doctor result was run by this content task.

Manual source installation ↗

Complete operator steps ↗

Source notes

packages/kiwicaptcha/integrations/symfony/docs/getting-started.md

packages/kiwicaptcha/integrations/symfony/docs/configuration.md

packages/kiwicaptcha/integrations/symfony/tests/Kernel/NewIntegratorSmokeKernel.php

packages/kiwicaptcha/integrations/symfony/tests/NewIntegratorSmokeTest.php

core

Keep verification server-side.

The PHP core is framework neutral. Rust and PHP share the record and risk-state contracts described by the repository. The host owns scope, storage, the expected transaction context and the outcome of the protected action.

Do not put the signing secret in a browser or native client. A client onVerify callback delivers a response; the server still decides.

Rust core ↗

PHP core ↗

Source notes

README.md

packages/kiwicaptcha/README.md

packages/kiwicaptcha-php/README.md

packages/kiwicaptcha/integrations/symfony/docs/claims-registry.md

server / sdks

Portable verification clients.

The SDK READMEs document store adapters, host middleware, outcome clients and doctor entry points. Publication syntax in a README is not a registry availability assertion on this website.

Check the algorithm and execution policy before issuing a record that a particular runtime cannot verify locally.

SDK	Host surfaces in source	Capability note
Go	Native verifier and middleware	Argon2id through x/crypto; execution-armed records have explicit refusal/delegation policy.
Node/TypeScript	Middleware, optional stores and outcome client	README documents SHA-256/RSW native proof verification and unsupported Argon parameters.
Python	Host middleware, stores, doctor and outcomes	argon2-cffi is a current core dependency; unusable binding requires the documented refusal/ sidecar path.
Ruby	Rack, Rails and Sinatra integration surfaces	Source-path installation is documented pending publication; retain the SDK execution/algorithm contract.
JVM	Servlet and Spring Boot surfaces; other host extension points	Consult store, algorithm and execution-policy scope before arming requests.
NET	ASP.NET middleware, Razor and Blazor surfaces	Verification client; not a separate complete enforcement plane.
Elixir	Plug and Phoenix surfaces	Source-path installation is documented pending publication; backend capability scope remains explicit.

Go source guide ↗

Node source guide ↗

Python source guide ↗

Ruby source guide ↗

Jvm source guide ↗

.NET source guide ↗

Elixir source guide ↗

Source notes

README.md

packages/kiwicaptcha-go/README.md

packages/kiwicaptcha-node/README.md

packages/kiwicaptcha-python-sdk/README.md

packages/kiwicaptcha-ruby/README.md

packages/kiwicaptcha-jvm/README.md

packages/kiwicaptcha-dotnet/README.md

packages/kiwicaptcha-elixir/README.md

---

browser

A shared browser lifecycle.

React, Vue, Svelte, Solid and Angular wrap the framework-free client core. Each render creates fresh markup; event-driven callbacks and teardown preserve the host-owned container.

Configure endpoint, sitekey, scope and language. The common callbacks handle the documented lifecycle. New step-up demand is a driver event; do not invent an onStepUp prop that the wrappers do not export.

The shared client shape

```
import { loadKiwiCaptcha, renderKiwiCaptcha } from "@kiwicaptcha/client-core";

const api = await loadKiwiCaptcha({ scriptSrc: "/kiwi.js" });
const handle = renderKiwiCaptcha(document.getElementById("mount")!, {
  endpoint: "/api/kcaptcha/challenge",
  sitekey: "pk-live-1",
  scope: "login",
  lang: "en",
  onVerify: (token) => submit(token),
  onError: (message) => show(message),
  onExpire: () => handle.reset(),
});
```

Copy example

The current client-core README's four-setting example, with its application callbacks. submit/show and the served script path belong to the host application; a callback token must still be verified by the server.

Browser integration contract, not a standalone executable or a package-publication assertion.

Shared client core ↗

React ↗

Vue ↗

Svelte ↗

Solid ↗

Angular ↗

Source notes

packages/kiwicaptcha-client-core/README.md

packages/kiwicaptcha-client-core/src/widget.ts

packages/kiwicaptcha-react/README.md

packages/kiwicaptcha-vue/README.md

packages/kiwicaptcha-svelte/README.md

packages/kiwicaptcha-solid/README.md

packages/kiwicaptcha-angular/README.md

packages/kiwicaptcha-wasm/assets/widget-driver.js

native

Client work outside the browser.

The repository also carries native status/solver integrations. Their backend still runs verification, and their native dispatch and execution limits are part of the contract.

An available source package is not a promise that every native framework, module or distribution channel is ready for an arbitrary deployment.

Client	Source interface	Important scope
Android	Compose and classic View over background JVM/Kotlin solving	No WebView; backend SiteVerify owns acceptance.
iOS	SwiftUI/UIKit and Swift solver package	Native solver and lifecycle; inspect exact challenge capability limits.
Flutter	Material widget with Rust FFI plus low-difficulty Dart path	Required native/FFI path applies above the supported low ceiling.
React Native	JS orchestration and native module contract	Harder work requires native module; Expo development build differs from Expo Go; execution-armed requests refused.

Android guide ↗

iOS guide ↗

Flutter guide ↗

React Native guide ↗

Source notes

packages/kiwicaptcha-android/README.md

packages/kiwicaptcha-ios/README.md

packages/kiwicaptcha-flutter/README.md

packages/kiwicaptcha-react-native/README.md

sidecar

A local verifier when you need it.

The verifier binary exposes a provider-shaped local HTTP or Unix-socket surface. Hosts without a native SDK, or a documented delegated dimension, can use that contract.

Pass the independent expected application binding when the operation requires it. Local delegation does not remove scope, lifecycle or storage duties.

Verifier sidecar guide ↗

Source notes

packages/kiwicaptcha-verifier/README.md

packages/kiwicaptcha-go/README.md

packages/kiwicaptcha-python-sdk/README.md

migration

Migrate the lifecycle. Review the differences.

The same-origin compatibility loader and SiteVerify surface cover the documented provider API subset. reCAPTCHA v3 scores are intentionally not invented, and provider-specific shapes/edge behavior outside the contract are not emulated.

Keep your backend's expected scope/action and policy authoritative. TTL parity is a deliberate configuration decision, not proof of universal drop-in behavior.

Migration guide ↗

SiteVerify contract ↗

Source notes

packages/kiwicaptcha/integrations/symfony/docs/migration.md

packages/kiwicaptcha/integrations/symfony/docs/siteverify.md

status

Read source status before installation.

At this reviewed snapshot the Rust core and Symfony bundle are explicitly pre-publication, and the Flex recipe is pending. Some SDKs document source-path installation; other READMEs show intended registry syntax.

Use the linked source guides and current publication status. This site does not offer a registry-install CTA or call an old GitHub runtime set the current source build.

Source notes

README.md

packages/kiwicaptcha/integrations/symfony/docs/getting-started.md

packages/kiwicaptcha-ruby/README.md

packages/kiwicaptcha-elixir/README.md

KC4_RELEASES

Download the complete website PDF ↓ All questions and answers →

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.

Also in the family: Faktor · Kiwi CI

kiwi captcha

SECURITY / EXPLICIT DECISIONS

Friction, then enforcement.

The mechanism is public. Your application owns the policy and the final action. Keep the proof, factor and authorization boundaries explicit.

model

A bounded anti-abuse model.

Computational work prices an attempt, not a person. Outsourced solvers, device pools and automation can pay it. Timing, decoys, IP binding and client evidence stay supplementary.

The current complete plane adds configured step-up, quarantine and denial. It still cannot rescue a compromised application, Redis instance or page script, or replace your transport and authentication architecture.

Source notes

SECURITY.md

README.md

profiles

Choose the posture your deployment can complete.

Profiles fill derived settings as the lowest-precedence layer. Explicit values in any layer win, within the forced privacy and validation rules; explicit null can clear a profile.

Use the source code and installed doctor for effective configuration. The current numeric documentation table has drift from profile code, so this website describes purpose rather than restating it as an exact matrix.

Profile	Purpose	Operator implication
balanced	Neutral documented baseline	Risk and factor setup are not automatically a full abuse posture.
privacy_strict	Strongest named privacy posture	Drops nonce-IP binding and forces sensitive evidence off; accept relay-mitigation tradeoff.
high_abuse / abuse_first	Same adaptive abuse posture	Wire its datasets, factors, real delivery, routes, scope/operation and protocol prerequisites.
compatibility	Documented provider/lifecycle migration posture	Binding/risk/features differ; no universal provider-score emulation.

Profile	Purpose	Operator implication
ha_safe	Pinned-primary authority posture	Explicit initialization and supported Redis topology; no automatic production pin.

Profile code at reviewed HEAD ↗

Effective configuration reference ↗

Source notes

packages/kiwicaptcha/integrations/symfony/docs/configuration.md

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/ProtectionProfileDefaults.php

packages/kiwicaptcha/integrations/symfony/tests/ProtectionProfileLayeringTest.php

complete / carriers

A demanded factor must have a path to completion.

Current abuse defaults use post-credential assurance for admin login, an anonymous ownership carrier for signup and a named principal factor with operation identity for financial_action.

A blanket step-up minimum at issuance can block the visitor before any widget or credential flow. A carrier without real delivery, an available factor, reachable routes or a resolvable principal is incomplete. The current doctor checks those relationships.

Source notes

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/ProtectionProfileDefaults.php

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/Configuration.php

packages/kiwicaptcha/integrations/symfony/src/Command/KiwiCaptchaDoctorCommand.php

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/KiwiCaptchaExtension.php

operation

Authorize this action, not a reusable pass.

The shared finalizer distinguishes FactorVerified from AuthorizedStepUp. It reproduces operation bindings, acquires finalization, reassesses risk, settles terminal authorization and only then journals completion effects.

Sensitive operations need a server-owned logical identity and canonical protected contents. An authorized demand for one action must not become credit to authorize another, even when the payload looks identical.

Source notes

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/StepUpDemandFinalizer.php

packages/kiwicaptcha/integrations/symfony/src/Validator/Constraints/KiwiCaptchaValidator.php

packages/kiwicaptcha/integrations/symfony/src/Controller/StepUpController.php

outcomes

Explain decisions without displaying identity material.

The configured risk explanation exposes names, action and work rung rather than pseudonym strings. Typed application outcomes provide confirmed feedback to the risk system; the application, not client telemetry, confirms legitimacy or abuse.

Outcome reporting and factor completion are not permission to bypass the authoritative request decision.

Source notes

packages/kiwicaptcha/integrations/symfony/docs/configuration.md

packages/kiwicaptcha/integrations/symfony/docs/risk-engine.md

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/StepUpDemandFinalizer.php

---

economics

Work cannot carry every stake.

The dedicated economic contract reports default browser-feasible work below declared abuse value. High-value policy needs a complete enforcement answer, not an invented claim that a few more bits make all abuse unprofitable.

The sequential extension still has an open browser/native same-hardware measurement. No speed ratio, savings statistic or hardware-cost promise is supplied by this identity or website.

Cost-to-abuse analysis ↗

Source notes

docs/cost-to-abuse.md

packages/kiwicaptcha/integrations/symfony/src/Command/KiwiCaptchaDoctorCommand.php

resources

Protect the verifier as well as the form.

Rate-limit issuance, bound outstanding challenges and cap verification concurrency. Memory-hard verification needs a fixed resource envelope and admission control.

The security Redis is authoritative state, not a disposable cache. Preserve noeviction, TTL margin and the replication/promotion contract. A verified acknowledgement alone does not control which replica a failover manager promotes.

Deployment and capacity guide ↗

Source notes

SECURITY.md

packages/kiwicaptcha/integrations/symfony/docs/operations.md

packages/kiwicaptcha/integrations/symfony/docs/configuration.md

assets

Serve one coherent runtime set.

Driver, worker, lazy modules, glue and solver identities are coupled. Mixed versions enter the controlled mismatch path; they must not silently produce a weaker accepted solution.

Use the asset delivery tier's CSP and immutable/SRI contract. A versioned source branch is not permission to mix it with an older GitHub release's assets.

Source notes

SECURITY.md

packages/kiwicaptcha-wasm/assets/widget-driver.js

KC4_RELEASES

evidence

Evidence has a date and a source fingerprint.

All seventeen current THREATS rows are marked RED because their recorded fingerprints are stale. They do not establish current HEAD coverage. That freshness refusal must not be shown as either a current GREEN claim or seventeen newly demonstrated attacks.

An independent external security audit remains a documented milestone. This website source review runs no new campaign, claims no full security audit and adds no certification seal.

Current threat ledger ↗

Security policy and reporting ↗

Source notes

THREATS.md

SECURITY.md

reporting

Use the private security-reporting route.

The repository security policy asks for suspected vulnerabilities through GitHub Security Advisories, with affected component/version and reproduction details. Artifact versions and repository snapshots are different identities.

Read the maintained reporting and supported-version policy before a rollout or disclosure.

Security policy ↗

Source notes

SECURITY.md

Download the complete website PDF ↓ All questions and answers →

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

kiwi captcha

PRIVACY / A FIRST-PARTY CONTRACT

Know what the check keeps.

First-party delivery and telemetry-off defaults are concrete properties. Adaptive continuity and optional factors have their own scope, configuration and retention.

defaults

Strict mode is the default.

The authoritative privacy document defines strict and standard modes. Strict forces telemetry off, same-origin enforcement and the documented timing/context restrictions. Standard honors explicit choices for the keys it governs.

Do not confuse privacy_mode: strict with the named privacy_strict protection profile. The latter also drops the nonce-bound IP tag; mode strict alone does not force binding_mode to none.

Setting	Default/contract	What it does not mean
privacy_mode: strict	Default mode; forces documented privacy-sensitive settings	Not the same as the separate privacy_strict profile.
telemetry: off	Default; strict mode keeps it off	Standard-mode deliberate opt-ins exist.
nonce-IP binding	Per-challenge keyed tag where configured	Relay mitigation, not a stable person identifier or universal guarantee.
privacy_strict profile	Drops binding tag and strongest evidence surfaces	Tradeoffs matter; policy still needs an operational contract.

Source notes

packages/kiwicaptcha/integrations/symfony/docs/privacy.md

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/Configuration.php

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/ProtectionProfileDefaults.php

signals

Supplementary signals stay supplementary.

Standard-mode minimal/full telemetry is client-controlled and forgeable. Coarse client context requires explicit server and widget opt-in, with the strict contract refusing it.

The documented coarse tag excludes canvas, audio, font-list and GPU fingerprints. Experimental challenge-scoped layout evidence has a narrower contract and is disabled under strict. None is proof of a browser or a person.

Source notes

packages/kiwicaptcha/integrations/symfony/docs/privacy.md

packages/kiwicaptcha/integrations/symfony/docs/claims-registry.md

SECURITY.md

continuity

Continuity is deliberate linkability.

Adaptive risk can set an HttpOnly first-party continuity cookie containing a random nonce. The engine stores its keyed pseudonym, not the raw cookie value.

Source and subnet pseudonyms rotate with epochs. Session and principal lifetimes differ, and linkability inside an epoch exists for rate limiting. Do not say every pseudonym rotates per request.

Source notes

packages/kiwicaptcha/integrations/symfony/docs/privacy.md

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/Configuration.php

retention

Retention is configured by state type.

The values below are current source defaults or named lifecycle rules, not a universal promise for every deployment. Inspect the effective configuration and overrides.

Security state must expire in ways that preserve its replay guards. A browser session cookie's life and the server session-state TTL are different settings.

State	Default or contract	Boundary
Rate-limit pseudonym rotation	3600 seconds by default	Rotation is not identical to counter retention; within-epoch linking remains.
Risk source/subnet epochs	900 seconds by default	Key identity rotation, not an erasure claim for all associated state.
Live risk counters	1800-second default TTL	Configured state_ttl_secs.
Server session risk/context state	1800-second default TTL	Separate session_state_ttl_secs; not derived from cookie lifetime.
Principal counters	86400-second default TTL	Configured principal_ttl_secs; principal is keyed before storage.
Challenge/consumed outcomes	Configured validity plus required guard retention	Keep TTL margin and durability contracts, not early deletion that reopens replay.
Marks and factor/demand records	Their own configured TTL/lifecycle	Do not substitute the live-counter TTL for all long-memory or factor state.

Source notes

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/Configuration.php

packages/kiwicaptcha/integrations/symfony/docs/privacy.md

packages/kiwicaptcha/README.md

SECURITY.md

address / factor

An address check has an address-delivery step.

Anonymous ownership receives the presented address to send the code through the application's sender. Stored protocol address material is HMAC-pseudonymous and the code is keyed-hashed; the grant binds the demanded scope and session context.

The actual mail transport, host request logs and application storage have their own data scope. First-party runtime does not mean an email address was never supplied or that every mail provider shares the component's retention contract.

Source notes

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/EmailOwnershipStepUp.php

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/EmailOwnershipCodeSenderInterface.php

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/KiwiCaptchaExtension.php

runtime

Keep the network claim specific.

The default widget issuance, same-origin runtime assets and token submission do not require a third-party runtime service. The claims registry records this path and its assumptions.

Your host page, proxy, transport and deliberately configured local sidecar or mail service remain separate decisions. KiwiCaptcha adds no network confidentiality beyond your application's HTTPS/TLS.

Source notes

packages/kiwicaptcha/integrations/symfony/docs/claims-registry.md

SECURITY.md

packages/kiwicaptcha-verifier/README.md

---

operations

Document the mode you actually deploy.

List your active profile/mode, adaptive continuity, factor delivery, pseudonymous state retention and operator overrides. Review application and infrastructure logging separately.

A source-linked privacy contract is more useful than a blanket 'no tracking of any kind' statement. Keep external-facing claims and their evidence aligned when behavior changes.

Authoritative privacy document ↗

Claims and assumptions ↗

Operator configuration ↗

Source notes

packages/kiwicaptcha/integrations/symfony/docs/privacy.md

packages/kiwicaptcha/integrations/symfony/docs/claims-registry.md

Download the complete website PDF ↓ All questions and answers →

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

DOCUMENTATION / AN OPERATOR PATH

Configure a flow people can finish.

Start with the real baseline. Keep the server authoritative. Add the complete factor, delivery and operating contract when your policy asks for more.

start

Start from the current source integration.

The reviewed README and manual guide still state that Symfony registry publication and the Flex recipe are pending. Use the repository bundle and its documented manual-equivalent installation path; this site does not pretend a one-command registry install is available.

Register BelConsulting\KiwiCaptchaBundle\KiwiCaptchaBundle, provide a strong signing secret and set the deployment's canonical HTTPS origin and Redis DSN through its environment.

Manual bundle registration

```
return [
    // ...
    BelConsulting\KiwiCaptchaBundle\KiwiCaptchaBundle::class => ['all' => true],
];
```

Copy example

From the manual installation guide. This is part of the source-install path while registry publication/Flex recipe remain pending.

Integrating application's config/bundles.php; require the repository bundle and its dependencies first.

Generate the signing secret

```
openssl rand -hex 32
```

Copy example

The getting-started guide's command. Store the generated value in the deployment environment; it does not belong in the widget.

Operator shell. This content task did not run or retain a generated secret.

Manual source guide ↗

Choose the integration layer ↗

Source notes

README.md

packages/kiwicaptcha/integrations/symfony/docs/getting-started.md

baseline

Configure the documented balanced baseline.

Set protection_profile, secret_key, public_base_url and redis_dsn. The DSN builds the Redis-backed storage and controls, while explicit service ids remain the advanced override path.

This snippet is present in the current verified-flow docs and smoke fixtures. Its existence is source evidence, not a claim that this research ran a deployment or your environment is ready.

Minimal Symfony configuration

```
kiwi_captcha:
    protection_profile: balanced
    secret_key: '%env(KIWI_SECRET_KEY)%'
    public_base_url: '%env(KIWI_PUBLIC_URL)%'
    redis_dsn: '%env(KIWI_REDIS_DSN)%'
```

Copy example

The repository's verified-flow example. This is the balanced baseline, not a complete abuse_first enforcement deployment.

Server configuration after manual repository installation; environment values must be supplied by the application.

Source notes

packages/kiwicaptcha/integrations/symfony/docs/configuration.md

packages/kiwicaptcha/integrations/symfony/tests/Kernel/NewIntegratorSmokeKernel.php

packages/kiwicaptcha/integrations/symfony/tests/NewIntegratorSmokeTest.php

packages/kiwicaptcha/integrations/symfony/docs/getting-started.md

widget / setup

Render the component. Verify its response.

The form type and Twig theme provide the component markup and configured runtime delivery.

The framework-free client example shows the endpoint, public sitekey, scope and language shape for client wrappers.

Keep the hidden response/binding fields and the current runtime contract intact. An onVerify token or ownership grant must go to the authoritative host verification path; it is not an access grant.

Register the form theme

```
twig:
    form_themes:
        - '@KiwiCaptcha/form_div_layout.html.twig'
```

Copy example

The real bundle theme supplies the component markup and runtime assets according to the configured delivery tier.

`Integrating Symfony application's Twig configuration.`

The shared client shape

```
import { loadKiwiCaptcha, renderKiwiCaptcha } from "@kiwicaptcha/client-core";

const api = await loadKiwiCaptcha({ scriptSrc: "/kiwi.js" });
const handle = renderKiwiCaptcha(document.getElementById("mount")!, {
  endpoint: "/api/kcaptcha/challenge",
  sitekey: "pk-live-1",
  scope: "login",
  lang: "en",
  onVerify: (token) => submit(token),
  onError: (message) => show(message),
  onExpire: () => handle.reset(),
});
```

Copy example

The current client-core README's four-setting example, with its application callbacks. submit/show and the served script path belong to the host application; a callback token must still be verified by the server.

`Browser integration contract, not a standalone executable or a package-publication assertion.`

Source notes

packages/kiwicaptcha/integrations/symfony/docs/getting-started.md

packages/kiwicaptcha/integrations/symfony/src/Form/Type/KiwiCaptchaType.php

packages/kiwicaptcha-client-core/README.md

packages/kiwicaptcha-wasm/assets/widget-driver.js

adaptive

Complete the abuse posture before relying on it.

high_abuse and abuse_first are the same current source posture. They enable adaptive risk, novelty enforcement and additional assurance with scope-specific carriers.

Current prerequisites include the relevant local ASN dataset and RSW configuration, real code delivery, step-up routes and supported factors, firewall→scope mapping, outcome bridge, protocol floors and authoritative operation identity where demanded. A profile name cannot supply an email address, a mail transport or your transaction identity.

1. Use stable scope identity and a server-owned allowlist. Map each protected login firewall through risk.login_scopes; main→login is the abuse-profile convention, not a universal firewall name.
2. Wire the actual principal/security events and outcome scope. Several different login scopes need an explicit bridge decision; do not claim a single inferred scope always exists.
3. Enable the carrier the scope names and an enrollment/bootstrap path legitimate users can complete.
4. Use a real sender or the inspected Mailer adapter. Resolve a username principal's address via its user model/interface service.
5. Import or register the application-facing begin/complete paths and apply the correct access control.
6. Sensitive actions place a server-owned logical id under KiwiCaptchaValidator::OPERATION_ID_ATTRIBUTE and supply canonical protected contents.
7. Deploy coherent binaries/assets and align central floors with the armed writer contract.
8. Meet the installed doctor's abuse-profile ASN and RSW requirements, with keys and local data maintained as documented.

Source notes

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/ProtectionProfileDefaults.php

packages/kiwicaptcha/integrations/symfony/src/Command/KiwiCaptchaDoctorCommand.php

packages/kiwicaptcha/integrations/symfony/tests/Kernel/NewIntegratorSmokeKernel.php

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/Configuration.php

delivery

Know the delivery and recipient seams.

Principal codes use risk.step_up.handlers.email_otp.sender; anonymous ownership uses risk.step_up.email_ownership.sender. Their interfaces serve different address contexts.

The current extension checks an available mailer.mailer service or a configured sender. Production refuses logging-only delivery for an enabled code factor. PrincipalEmailResolverInterface is the inspected optional service seam; this website does not invent an email_resolver or mailer_from config key.

Purpose	Actual seam	Responsibility
Principal code	StepUpCodeSenderInterface / handlers.email_otp.sender	Deliver to the resolved account address; keep factor secret server-side.
Anonymous address ownership	EmailOwnershipCodeSenderInterface / email_ownership.sender	Deliver to the presented address only under a valid demand.
Custom user model address	PrincipalEmailResolverInterface service	Resolve a usable email from the authenticated user; null refuses delivery.

Purpose	Actual seam	Responsibility
Automatic mailer adapter	mailer.mailer service inspected by extension	Recipient/transport must actually work; current from address derives from canonical-origin host.

Source notes

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/KiwiCaptchaExtension.php

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/Configuration.php

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/PrincipalEmailResolverInterface.php

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/EmailOwnershipCodeSenderInterface.php

routes

Register application-facing factor routes.

The default challenge route loader does not register the step-up flow. The shipped routing resource provides the four current begin/complete routes for an application import.

Keep principal begin/complete in the application's authentication context and make the anonymous ownership carrier reachable to the demanded visitor. Paths and access control belong to the host flow.

Import the application-facing step-up routes

```
kiwi_captcha_step_up:
    resource: '@KiwiCaptchaBundle/Resources/config/routes/step_up.php'
```

Copy example

Exact import from the current routing resource. These routes are outside the default challenge loader: the application owns their paths, chrome and access control.

Integrating application's config/routes.yaml, when the step-up plane is enabled and access control is configured.

Source notes

packages/kiwicaptcha/integrations/symfony/src/Resources/config/routes/step_up.php

packages/kiwicaptcha/integrations/symfony/src/Controller/StepUpController.php

---

operate

Operate the state and resource contract.

Use production storage with the concurrency semantics your guarantee requires. Redis is authoritative state: noeviction, bounded capacity, TTL margins and replay-safe promotion matter. APCu is a single-PHP-host adapter; its CLI segment acknowledgment is explicit and it is not a replicated fleet store.

Configure trusted proxies rather than attributing every client to the proxy address. Separate signing-key rotation from stable abuse-identity secrets. Limit issuance, outstanding work and Argon verification concurrency against the actual memory budget.

Source notes

SECURITY.md

packages/kiwicaptcha/integrations/symfony/docs/configuration.md

packages/kiwicaptcha/integrations/symfony/src/DependencyInjection/Configuration.php

packages/kiwicaptcha/integrations/symfony/docs/operations.md

packages/kiwicaptcha/integrations/symfony/docs/security-hardening.md

csp / assets

Match CSP to the asset delivery tier.

The default files tier uses same-origin immutable/SRI assets and a same-origin worker. The inline compatibility tier has a different worker directive. Argon2id requires its supported worker/WASM path; the documented SHA fallback does not make every profile interchangeable.

Serve the coupled driver, modules, worker, glue and solver from one coherent build. Mismatch clears the response rather than silently accepting stale output. The CLI doctor cannot fully observe your live page CSP, so its warning is not a browser pass.

CSP/worker requirements ↗

Delivery configuration ↗

Source notes

SECURITY.md

packages/kiwicaptcha/integrations/symfony/docs/getting-started.md

packages/kiwicaptcha-wasm/assets/widget-driver.js

packages/kiwicaptcha/integrations/symfony/src/Command/KiwiCaptchaDoctorCommand.php

doctor

Inspect what is actually installed.

Run kiwicaptcha:doctor inside the integrating application. The command reports individual checks and exits nonzero for FAIL. WARN is not a fabricated PASS; it names an unresolved observation or deployment choice.

Current checks cover storage/Redis, secrets/keyring/origin, client-IP policy, protocol writers/floors, admission, datasets, login/outcome/carrier delivery/routes, sensitive-operation policy and additional profile prerequisites. Use the complete report with the linked contract.

Inspect the installed wiring

```
bin/console kiwicaptcha:doctor
```

Copy example

The command inspects the services and configuration actually installed in the application. A FAIL returns a nonzero exit; WARN names something the command could not establish or a deployment decision still open.

Inside the integrating Symfony application. No application deployment or doctor result was run by this content task.

Source notes

packages/kiwicaptcha/integrations/symfony/src/Command/KiwiCaptchaDoctorCommand.php

packages/kiwicaptcha/integrations/symfony/docs/configuration.md

packages/kiwicaptcha/integrations/symfony/docs/security-hardening.md

---

recover

Give each failure the right recovery.

Expired credentials clear only the check's response/binding; preserve host form data and permit reacquisition. Terminal flow failure offers native Retry. Temporary unavailable verification stays retryable rather than a guessed invalid result.

A worker unavailable state does not silently weaken memory-hard work. A version mismatch needs coherent assets/reload. STEP_UP_REQUIRED is a policy demand with a carrier path, not an endless retry spinner.

View the lifecycle specimen ↗

Source notes

packages/kiwicaptcha-wasm/assets/widget-driver.js

packages/kiwicaptcha/integrations/symfony/docs/siteverify.md

packages/kiwicaptcha/integrations/symfony/docs/security-hardening.md

packages/kiwicaptcha/integrations/symfony/Resources/ACCESSIBILITY.md

faq

Questions before deployment.

These answers distinguish inspected behavior from proposed website presentation. Read the linked source snapshot when your configuration or product version changes.

Does KiwiCaptcha prove that a visitor is human?

No. Computational work proves a selected computation under its challenge rules. Automated clients can pay that work too. Email ownership establishes control of an address; a principal factor supports the application's configured assurance. None is a universal proof of humanness.

What is the complete first-class integration today?

Symfony is the current complete enforcement plane: issuance, risk decisions, step-up, quarantine and denial. Other language SDKs are verification/risk clients and middleware with their own capability contracts.

Does a completed widget authorize the request?

No. The browser has a response token or, in the ownership path, a grant. The application must verify the response and its lifecycle. A factor must also pass the demand finalizer before an operation or pending login is authorized.

Why can the application ask for another check?

A scope's value and risk policy can require additional assurance. Anonymous address ownership and principal-bound factors have different carriers. The application owns that decision; the PoW widget does not ask an arbitrary 'I am human' checkbox.

Can raw work price out valuable abuse?

The published default-rung analysis says it cannot at the browser-feasible costs measured. Use a scope-appropriate, completable step-up or denial policy, and bound issuance and verification resources.

Are abuse_first and high_abuse different profiles?

They are two accepted names for the same current source defaults and composition. Choose one spelling consistently. An explicit setting in any configuration layer still takes precedence, subject to strict-mode and validation contracts.

Can I enable an abuse profile with only four config keys?

The four-key balanced example is the starting baseline. A complete abuse posture has additional requirements: current source checks cover a fresh local ASN dataset, pre-armed RSW configuration, delivery, routes, factors, login scope mapping, outcome bridge, protocol floors and sensitive-operation identity. Let the installed doctor show the effective requirements.

Should every valuable scope use minimum: step_up?

Do not apply it blindly. Current code explicitly notes that a step-up issuance minimum blocks a visitor before the widget runs. Login uses its post-credential policy; anonymous signup uses a completable ownership carrier; sensitive actions need their named factor and server-owned operation binding.

How does the email factor deliver a code?

Through your application's real sender. The bundle uses an available Symfony Mailer service or a configured sender implementation. Production refuses an enabled factor left on the logging development sender. A username principal must resolve to a deliverable address.

Which sender and resolver seams are supported?

Principal codes use risk.step_up.handlers.email_otp.sender with StepUpCodeSenderInterface. Anonymous ownership uses risk.step_up.email_ownership.sender with EmailOwnershipCodeSenderInterface. PrincipalEmailResolverInterface is the inspected optional service seam; the current configuration tree does not contain an email_resolver key.

What does single-use mean during retries?

Atomic storage permits one consume/derivation. A retained deterministic result can be recovered for the exact original logical operation. It is not permission for a different operation to redeem the token again.

How are sensitive operations distinguished?

The host supplies its own logical operation identity and canonical protected contents. Demand finalization checks both identity and content digests; the same payload under a different logical operation is a different action. A raw client correlation value alone is not server authority.

Does privacy_mode: strict disable every IP-derived tag?

Strict mode forces telemetry and selected privacy-sensitive behavior; it does not itself force binding_mode off. The separate privacy_strict protection profile drops the nonce-bound IP tag. The tradeoff is reduced IP relay mitigation.

What data does adaptive risk keep?

The configured engine uses keyed pseudonymous state and a first-party continuity cookie. Source/subnet identities rotate with epochs; session and principal lifetimes differ. Counter, mark and factor retention is configured. No raw-IP persistence claim should be extended to your application's own logs or mail system.

Are third-party runtime services required?

The default widget path uses the application origin for issuance, runtime assets and token submission. Host-page code, deployment endpoints, an explicitly chosen mail provider or selected local SDK sidecar have their own scope; the claim is specific to the default widget execution path.

Which languages does the widget ship?

English is eager. The lazy module supplies German, French, Spanish, Italian, Dutch, Polish, Portuguese, Arabic and Brazilian Portuguese. That is ten locale packs across nine languages plus a regional variant. New step-up strings at this snapshot are English-only and should not be advertised as fully translated in every pack.

What happens when work expires or fails?

Expiry clears the credential and binding, preserving the host form entries. Recoverable failure offers native Retry. Memory-hard worker failure does not silently weaken the work. A solver version mismatch needs a coherent asset set and reload.

Is the new ownership dialog just another failure state?

No. STEP_UP_REQUIRED is a typed policy demand. The ownership carrier has its address/code dialog; a principal-carried demand raises kiwi:step-up-required for the application's authentication flow. Neither is an arbitrary CAPTCHA puzzle.

Do all SDKs verify every algorithm locally?

No. Consult the actual capability contract. For example, Node documents native SHA-256/RSW verification and refuses unsupported Argon2 parameters; Python's current core dependency includes argon2-cffi, with explicit admission/delegation rules when unavailable. Execution evidence may need the local verifier sidecar.

Is the provider compatibility bridge universal?

It covers the documented reCAPTCHA, hCaptcha and Turnstile lifecycle subsets and SiteVerify migration. Provider-specific v3 scores, exact token shapes and behavior outside the stated surface must move onto your own verification and policy.

What do the current red-team rows establish?

At the reviewed HEAD all seventeen THREATS rows are RED because their recorded source fingerprints are stale. Those runs do not establish current coverage. This is a freshness refusal, not a statement that seventeen new attacks succeeded. Prior GREEN results are preserved as historical evidence with their original scope.

What accessibility evidence is available?

The source documents automated component checks and a separate release-specific assistive-technology qualification gate. The passive work check avoids a required cognitive test. That is not automatic host-site WCAG/EAA conformance, and the new factor flow needs its own scope rather than inheriting an unqualified claim.

Can I install a current release from a registry?

The reviewed README and Symfony guide still state that Rust/Symfony registry publication and the Flex recipe are pending. GitHub's latest listed snapshot is v1.6.31 from August 2026; it is not the current source tree. Use the source integration guides and check current release status before rollout.

Source notes

README.md

packages/kiwicaptcha/integrations/symfony/docs/claims-registry.md

reference

Use the owning contract.

The dedicated documents narrow broad top-level promises. Economics, privacy, claim assumptions, operational storage requirements and migration coverage each have an owning source.

Keep current source, dated identity analysis and published artifact status separate. Do not turn source inspection or historical campaigns into a new full-audit claim.

- Core ↗
- Configuration ↗
- Operations ↗
- SiteVerify ↗
- Troubleshooting ↗
- Claims registry ↗
- Migration ↗

Source notes

README.md

docs/cost-to-abuse.md

packages/kiwicaptcha/integrations/symfony/docs/privacy.md

packages/kiwicaptcha/integrations/symfony/docs/claims-registry.md

SECURITY.md

KC4_RELEASES

Download the complete website PDF ↓ All questions and answers →

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

kiwi captcha

WIDGET / STATUS THAT MAKES SENSE

Show what is happening.

A static mark, readable words and an appropriate next step. The passive baseline and additional-factor demand are separate component jobs.

preview

Inspect the baseline check.

Change the state, language, host-container width and component text size. Labels should wrap and the component should grow; text is not hidden behind ellipses.

This specimen simulates presentation only. It computes no proof-of-work, issues no response token or grant and performs no network check.

State

Checking ˅

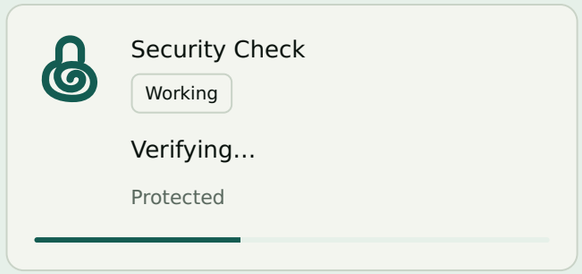
Language

English

Widget width

320 px

200% component text



Computational work is active. The track is decorative; no percentage is implied.

Source notes

packages/kiwicaptcha/design/README.md

packages/kiwicaptcha/resources/widget.css

packages/kiwicaptcha-wasm/assets/widget-driver.js

states

Ten source DOM states. More than one result kind.

pending is deferred execution that never started; idle is a manual reacquisition/reset state. Connecting and solving describe the flow, while done means a response is available.

Current done can follow work or the ownership grant path. The server still verifies context and the demand finalizer still authorizes. Expiry, failure, unavailable execution and mixed versions need distinct next steps.

State	Meaning	Recovery
pending	Deferred execution has never started.	No extra human control; application invokes execute.
idle	Manual reacquisition/reset state after a credential or flow existed.	Native Retry when provided by driver.
connecting	Acquiring a challenge.	Bounded flow recovery on failure.
solving	Selected computation in progress.	Driver governs completion/failure.
done	Browser response is available: a completed PoW token, or a completed anonymous email-ownership grant in the new step-up path.	Host continues to authoritative server verification; later expiry remains possible.
failed	Terminal flow failure after bounded recovery.	Native Retry. Two automatic retries precede terminal failure in driver.
expired	Credential validity ended.	Clear token/binding/aliases; preserve host form data; native Retry reacquires.
worker-unavailable	Required off-main-thread solver unavailable.	Native Retry can reacquire; no silent memory-hard downgrade/main-thread solve.
execution-unavailable	Required armed execution interpreter failed.	Native Retry reruns flow. No solution submitted without required evidence.

State	Meaning	Recovery
solver-mismatch	Worker/driver/glue protocol identities inconsistent.	Reload/update a coherent asset set; ordinary Retry cannot repair mixed assets. Never accept mismatched output.

Source notes

packages/kiwicaptcha-wasm/assets/widget-driver.js

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/StepUpDemandFinalizer.php

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/EmailOwnershipStepUp.php

stepup

A demand is not a puzzle or generic failure.

Current ownership demand can collect an address and then a code inside a labeled dialog. A principal-carried demand instead emits kiwi:step-up-required so the host authentication flow can route the user.

The source reuses solving or idle presentation for these views; it does not introduce an eleventh DOM state. New step-up strings at this snapshot are in eager English only, not the lazy locale rows.

Flow

Anonymous email ownership

View

Additional check requested

Explore the demanded factor separately from the passive computational status panel.

English visual concept. The newly added runtime step-up strings are not translated in every lazy locale pack.

Local design simulation only.

Email ownership / English design specimen

Additional check requested

The application demanded address ownership for this scope. No mail or token is created by this preview.

Continue preview

Source notes

packages/kiwicaptcha-wasm/assets/widget-driver.js

packages/kiwicaptcha-wasm/assets/widget-locales.js

packages/kiwicaptcha/integrations/symfony/src/Security/StepUp/EmailOwnershipStepUp.php

language

Let the words stay visible.

The runtime has ten packs: English plus nine lazy packs, spanning nine languages and Brazilian Portuguese. The older design reference exercises six of them.

Arabic sets lang and RTL direction. Long German/Portuguese labels are useful stress specimens. New factor translation scope must be described honestly rather than inheriting the baseline pack count.

Source notes

packages/kiwicaptcha-wasm/assets/widget-locales.js

packages/kiwicaptcha-wasm/assets/widget-driver.js

packages/kiwicaptcha/design/README.md

packages/kiwicaptcha/resources/widget.css

---

semantics

Meaningful states. Decorative motion.

The baseline group has readable status, one polite live region, a hidden decorative mark/track and a native recovery button. The track is not a slider, draggable task or determinate progressbar.

Keep visible focus and generous targets, reduced-motion support, forced-colors boundaries and natural text wrapping. Component text scaling is not the same evidence as actual user/browser zoom.

Source notes

packages/kiwicaptcha-client-core/src/markup.ts

packages/kiwicaptcha/resources/widget.css

packages/kiwicaptcha/integrations/symfony/Resources/ACCESSIBILITY.md

recovery

Recovery follows the failure.

The driver allows bounded automatic retries before terminal failure, then exposes native Retry. Expiry preserves host form data. Required execution/worker failure refuses rather than silently degrading the demanded proof.

Reload/update guidance belongs to a version mismatch. A simulated Reset preview button simply changes the specimen; it must not claim to repair a real worker or deployed factor.

Source notes

packages/kiwicaptcha-wasm/assets/widget-driver.js

SECURITY.md

host

Keep the host flow authoritative.

Framework callbacks deliver the response and lifecycle events through the shared client contract. Additional demand handling and final application decisions remain with the host.

The brand website self-hosts its fonts. The production component should not gain a brand-font request or sibling-product dependency.

Integration roles ↗

Operator guide ↗

Accessibility evidence ↗

Source notes

packages/kiwicaptcha-client-core/README.md

packages/kiwicaptcha-client-core/src/widget.ts

packages/kiwicaptcha/design/README.md

Download the complete website PDF ↓ All questions and answers →

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

KiwiCaptcha / Visual identity

Quiet, with character.

An open coil. Humanist lettering. A quieter rhythm.

The fixed logo masters



INK / PAPER Download SVG



TEAL / INK Download SVG

Fixed outlined artwork. Preserve letter spacing and H/4 clear space. Preferred symbol size: 24 px; tiny icons supplied separately.

Palette / Family foundations, own accent

Ink, Paper and Teal.

Ink	Forest	Paper	Sage	Teal	Teal Deep
#08110C	#143D2B	#F3F5EF	#92AA99	#9EDFD4	#145C52

Use the soft accent on Ink; the deeper accent for readable text on Paper. State colors keep their own semantic meaning.

Typography / Its own voice

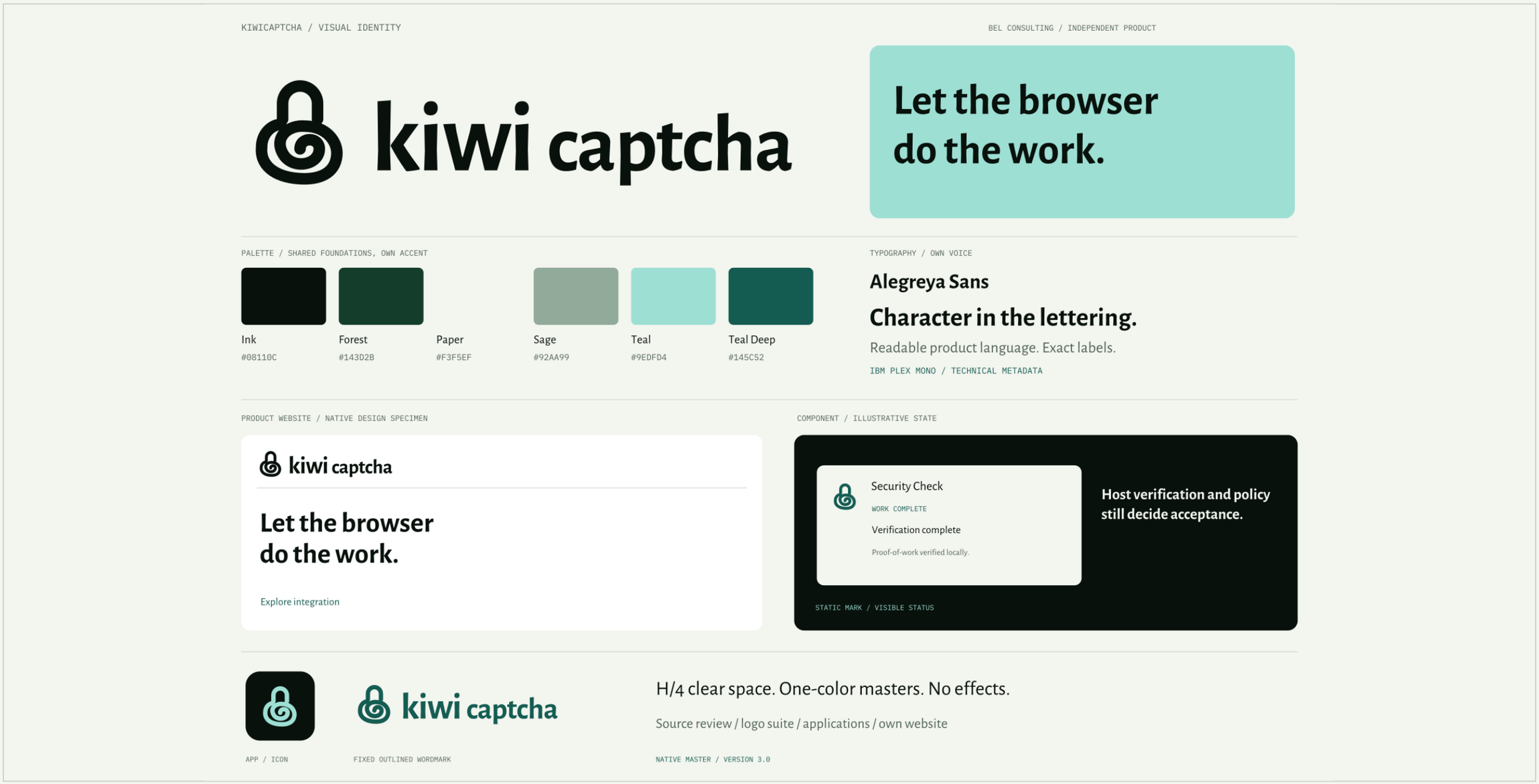
Let the browser do the work.

Alegreya Sans for headings and reading.

Humanist strokes, open rhythm.

IBM Plex Mono for technical metadata. The logo is fixed outlined artwork; host interfaces retain their own typography.

A complete application system.



Application concept. Use the native SVG masters for production artwork.

Downloads

Everything in its own project.

Logo suite, source analysis, usage rules, design tokens, font licenses, applications and generation briefs.

Visual identity guide	PDF
Complete design kit	ZIP
Standalone symbol	SVG
Stacked logo	SVG
Application icon	SVG
Design tokens	JSON

Current application package

The websites, in PDF form.

Complete website edition	PDF / ALL PRODUCT PAGES
Retained prior content	DATED RESEARCH & WEBSITE

identity

Distinct, with a family connection.

KiwiCaptcha owns its symbol, wordmark, colors, applications, guide and root website. Shared Ink, Forest, Paper and Sage provide family discipline; teal, humanist reading type and the form-first experience belong to this product.

The company endorsement and small related links are sufficient family context. A sibling product is not the home page, installation prerequisite or runtime asset host.

Source notes

README.md

packages/kiwicaptcha/resources/kiwi-mark.svg

packages/kiwicaptcha/design/README.md

---

master

Keep the canonical Spiral Lock exact.

The packaged resources/kiwi-mark.svg is the symbol authority. Keep both paths, 64×64 viewBox, 6.6-unit currentColor stroke, round caps/joins and no fill. The raised shackle stays still and separate from the open coil.

The bare mark leads branding and favicon use. The repository shield is a secondary status emblem, not an independent certification seal.

Source notes

packages/kiwicaptcha/resources/kiwi-mark.svg

packages/kiwicaptcha/design/README.md

packages/kiwicaptcha/tools/sync-brand.mjs

lettering

Use the artwork, not a typed approximation.

The humanist wordmark is fixed outlined artwork. Its spacing and joins belong to the master, and the captcha portion remains large enough to name the product at real sizes.

Alegreya Sans gives headings and reading texture their own rhythm; IBM Plex Mono stays with exact technical metadata. Do not rebuild the logo by choosing a font weight in CSS.

Source notes

packages/kiwicaptcha/design/README.md

suite

A complete logo and application suite.

Use the horizontal or stacked signature for primary introductions, the wordmark in known branded context, and the symbol for compact monochrome or icon slots.

The package should include currentColor/Ink/white/accent native SVGs, suitable PNG exports, favicon/app sizes, document/social artwork, tokens, fonts/licenses and reproduction proofs. Use the actual masters for production; concept images remain presentation artwork.

Source notes

packages/kiwicaptcha/resources/kiwi-mark.svg

packages/kiwicaptcha/design/README.md

use

Give the visible artwork room.

Measure clearspace from visible symbol/lettering, not empty viewBox padding. Preserve aspect ratio, fixed spacing and approved color versions.

Inspect the finished lettering at its actual minimum output. Native small-icon proofs should cover 16/24/32/48/64 px. Do not redraw a different micro symbol while claiming the exact canonical paths.

Keep the master intact: Stretching or rotation; Extra coils/ leaves or feature diagrams; Gradients, glow and shadows; Typed replacement wordmark; Fake compliance or human-verification badges

Source notes

packages/kiwicaptcha/resources/kiwi-mark.svg

packages/kiwicaptcha/design/README.md

color / type

Color has a job. Type has a voice.

Ink/Forest/Paper/Sage establish the family. KiwiCaptcha teal highlights dark surfaces; Deep Teal carries readable text on Paper. The soft teal is decorative on light backgrounds where its contrast is insufficient for text.

Use words and shapes alongside semantic color. Keep long labels readable and panel rhythm restrained; a background of repeated coils or generic floating cards competes with the actual mark.

Source notes

packages/kiwicaptcha/resources/widget.css

packages/kiwicaptcha/design/README.md

rollout

Update the generator before synchronizing.

The current product generator still emits an Inter live-text lockup. Adopt the approved humanist outline in that generator's lockup template before running sync/check; a direct edit to its generated lockup will be overwritten.

Preserve the symbol and its framework/template/compat mirrors. This identity handoff does not claim the source repository or runtime component has already been redesigned.

Regenerate canonical mirrors

```
node packages/kiwicaptcha/tools/sync-brand.mjs
node packages/kiwicaptcha/tools/sync-brand.mjs --check
```

Copy example

Run from the product repository root after an approved rollout. The current generator emits an Inter text lockup; adopt the supplied humanist outline in the generator template first, preserving the canonical mark.

Product-maintainer asset handoff. No product repository change performed by this task.

Source notes

packages/kiwicaptcha/tools/sync-brand.mjs

packages/kiwicaptcha/design/README.md

---

research / 2026 / 10 / 11

Preserved research / 11 October 2026.

The previous standalone analysis and guide are preserved in dated reference material at ebf81f4d33b0094aa8eca61bda7e08d8d522f8a5. Its product reasoning, logo rules, examples, state details and sources remain available with their original scope.

The current source update is not a silent replacement of that work. Changes in enforcement, factors, SDK capability and threat-ledger freshness are identified at 272f0ddb888f2ff35446667fcd8ffcb61a09af32.

Source notes

KC4_COMPARE

README.md

THREATS.md

Download the complete website PDF ↓ All questions and answers →

---

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

kiwi captcha

KiwiCaptcha / Questions and answers

The useful details, in one place.

Mechanism, deployment, factors, result lifecycle, privacy and product boundaries.

Find an answer

Does KiwiCaptcha prove that a visitor is human?

No. Computational work proves a selected computation under its challenge rules. Automated clients can pay that work too. Email ownership establishes control of an address; a principal factor supports the application's configured assurance. None is a universal proof of humanness.

What is the complete first-class integration today?

Symfony is the current complete enforcement plane: issuance, risk decisions, step-up, quarantine and denial. Other language SDKs are verification/risk clients and middleware with their own capability contracts.

Does a completed widget authorize the request?

No. The browser has a response token or, in the ownership path, a grant. The application must verify the response and its lifecycle. A factor must also pass the demand finalizer before an operation or pending login is authorized.

Why can the application ask for another check?

A scope's value and risk policy can require additional assurance. Anonymous address ownership and principal-bound factors have different carriers. The application owns that decision; the PoW widget does not ask an arbitrary 'I am human' checkbox.

Can raw work price out valuable abuse?

The published default-rung analysis says it cannot at the browser-feasible costs measured. Use a scope-appropriate, completable step-up or denial policy, and bound issuance and verification resources.

Are abuse_first and high_abuse different profiles?

They are two accepted names for the same current source defaults and composition. Choose one spelling consistently. An explicit setting in any configuration layer still takes precedence, subject to strict-mode and validation contracts.

Can I enable an abuse profile with only four config keys?

The four-key balanced example is the starting baseline. A complete abuse posture has additional requirements: current source checks cover a fresh local ASN dataset, pre-armed RSW configuration, delivery, routes, factors, login scope mapping, outcome bridge, protocol floors and sensitive-operation identity. Let the installed doctor show the effective requirements.

Should every valuable scope use minimum: step_up?

Do not apply it blindly. Current code explicitly notes that a step-up issuance minimum blocks a visitor before the widget runs. Login uses its post-credential policy; anonymous signup uses a completable ownership carrier; sensitive actions need their named factor and server-owned operation binding.

How does the email factor deliver a code?

Through your application's real sender. The bundle uses an available Symfony Mailer service or a configured sender implementation. Production refuses an enabled factor left on the logging development sender. A username principal must resolve to a deliverable address.

Which sender and resolver seams are supported?

Principal codes use risk.step_up.handlers.email_otp.sender with StepUpCodeSenderInterface. Anonymous ownership uses risk.step_up.email_ownership.sender with EmailOwnershipCodeSenderInterface. PrincipalEmailResolverInterface is the inspected optional service seam; the current configuration tree does not contain an email_resolver key.

What does single-use mean during retries?

Atomic storage permits one consume/derivation. A retained deterministic result can be recovered for the exact original logical operation. It is not permission for a different operation to redeem the token again.

How are sensitive operations distinguished?

The host supplies its own logical operation identity and canonical protected contents. Demand finalization checks both identity and content digests; the same payload under a different logical operation is a different action. A raw client correlation value alone is not server authority.

Does privacy_mode: strict disable every IP-derived tag?

Strict mode forces telemetry and selected privacy-sensitive behavior; it does not itself force binding_mode off. The separate privacy_strict protection profile drops the nonce-bound IP tag. The tradeoff is reduced IP relay mitigation.

What data does adaptive risk keep?

The configured engine uses keyed pseudonymous state and a first-party continuity cookie. Source/subnet identities rotate with epochs; session and principal lifetimes differ. Counter, mark and factor retention is configured. No raw-IP persistence claim should be extended to your application's own logs or mail system.

Are third-party runtime services required?

The default widget path uses the application origin for issuance, runtime assets and token submission. Host-page code, deployment endpoints, an explicitly chosen mail provider or selected local SDK sidecar have their own scope; the claim is specific to the default widget execution path.

Which languages does the widget ship?

English is eager. The lazy module supplies German, French, Spanish, Italian, Dutch, Polish, Portuguese, Arabic and Brazilian Portuguese. That is ten locale packs across nine languages plus a regional variant. New step-up strings at this snapshot are English-only and should not be advertised as fully translated in every pack.

What happens when work expires or fails?

Expiry clears the credential and binding, preserving the host form entries. Recoverable failure offers native Retry. Memory-hard worker failure does not silently weaken the work. A solver version mismatch needs a coherent asset set and reload.

Is the new ownership dialog just another failure state?

No. STEP_UP_REQUIRED is a typed policy demand. The ownership carrier has its address/code dialog; a principal-carried demand raises kiwi:step-up-required for the application's authentication flow. Neither is an arbitrary CAPTCHA puzzle.

Do all SDKs verify every algorithm locally?

No. Consult the actual capability contract. For example, Node documents native SHA-256/RSW verification and refuses unsupported Argon2 parameters; Python's current core dependency includes argon2-cffi, with explicit admission/delegation rules when unavailable. Execution evidence may need the local verifier sidecar.

Is the provider compatibility bridge universal?

It covers the documented reCAPTCHA, hCaptcha and Turnstile lifecycle subsets and SiteVerify migration. Provider-specific v3 scores, exact token shapes and behavior outside the stated surface must move onto your own verification and policy.

What do the current red-team rows establish?

At the reviewed HEAD all seventeen THREATS rows are RED because their recorded source fingerprints are stale. Those runs do not establish current coverage. This is a freshness refusal, not a statement that seventeen new attacks succeeded. Prior GREEN results are preserved as historical evidence with their original scope.

What accessibility evidence is available?

The source documents automated component checks and a separate release-specific assistive-technology qualification gate. The passive work check avoids a required cognitive test. That is not automatic host-site WCAG/EAA conformance, and the new factor flow needs its own scope rather than inheriting an unqualified claim.

Can I install a current release from a registry?

The reviewed README and Symfony guide still state that Rust/Symfony registry publication and the Flex recipe are pending. GitHub's latest listed snapshot is v1.6.31 from August 2026; it is not the current source tree. Use the source integration guides and check current release status before rollout.

Read the operator guide →

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

Research / Retained content

Current edition. Complete prior reference.

All earlier identity work and website content remain available, with their original source baselines clearly dated.

Current baseline / 12 October 2026

KiwiCaptcha at 272foddb888f

Symfony is the first-class enforcement plane; SDKs are verification/risk clients. Adaptive authorization and result kinds extend the earlier computational-only positioning. Campaign assurance records are stale at the current baseline.

Current source baseline: 272f0ddb888f2ff35446667fcd8ffcb61a09af32
Prior research baseline: ebf81f4d33b0094aa8eca61bda7e08d8d522f8a5

Prior website / 11 October 2026

The earlier edition, retained.

Index / Prior edition ↗

Integration / Prior edition ↗

Playground / Prior edition ↗

Brand / Prior edition ↗

The expanded identity guide retains every original page, image, construction rule, source analysis and generation brief. Dated corrections introduce the new edition.

kiwi captcha

Symfony-first adaptive anti-abuse. First-party control.

Source & documentation ↗

Identity & downloads

Research & prior edition

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

Prior edition / 11 October 2026 Retained design and pinned research. Current source and expanded experience are in the updated edition.

Open current website →

KiwiCaptcha / Visual identity

Quiet, with character.

An open coil. Humanist lettering. A quieter rhythm.

The fixed logo masters



INK / PAPER Download SVG



TEAL / INK Download SVG

Fixed outlined artwork. Preserve letter spacing and H/4 clear space. Preferred symbol size: 24 px; tiny icons supplied separately.

Palette / Family foundations, own accent

Ink, Paper and Teal.

Ink	Forest	Paper	Sage	Teal	Teal Deep
#08110C	#143D2B	#F3F5EF	#92AA99	#9EDFD4	#145C52

Use the soft accent on Ink; the deeper accent for readable text on Paper. State colors keep their own semantic meaning.

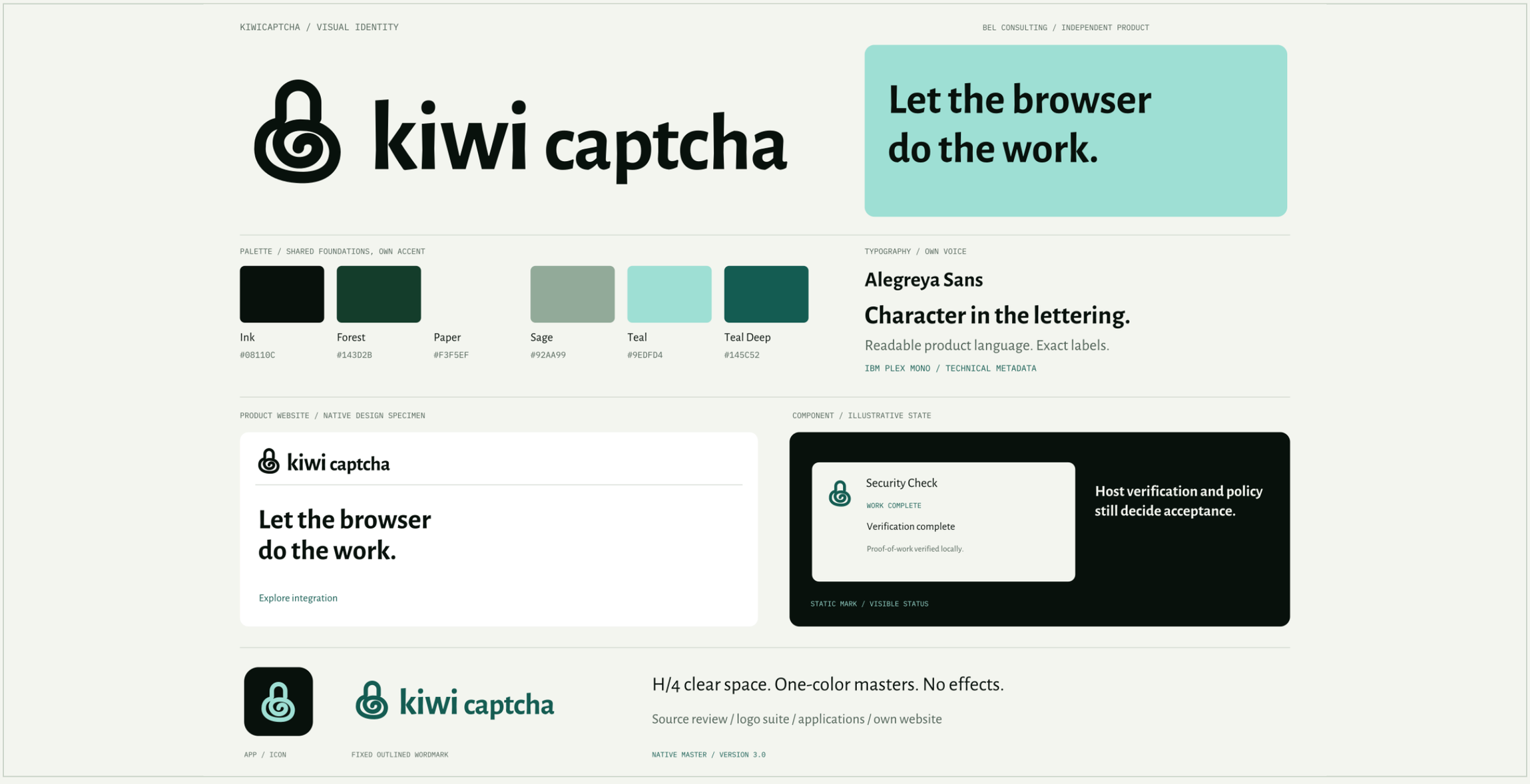
Typography / Its own voice

Let the browser do the work.

Alegreya Sans for headings and reading.
Humanist strokes, open rhythm.

The identity in use

A complete application system.



Downloads

Everything in its own project.

Logo suite, source analysis, usage rules, design tokens, font licenses, applications and generation briefs.

Visual identity guide	PDF
Complete design kit	ZIP
Standalone symbol	SVG
Stacked logo	SVG
Application icon	SVG
Design tokens	JSON

kiwi captcha

Self-hosted computational checks. First-party control.

Source & documentation ↗

Logo & design assets

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

kiwi captcha

Integration Widget Identity

Prior edition / 11 October 2026 Retained design and pinned research. Current source and expanded experience are in the updated edition. Open current website →

Computational checks / First-party control

Let the browser do the work.

A self-hosted check that fits quietly into your form. Your application owns the challenge, verification and acceptance policy.

Read the integration guide →

Try the widget

A form, with less interruption.

Email address

you@example.com

Security Check

Working

Verifying…

Protected

Illustrative host form / Computational work runs locally. The host verifies the result before acceptance.

Keep the check yours

Small on the page. Clear behind it.

First-party assets, explicit lifecycle rules and operator-controlled work. The visitor sees status and recovery.

Your runtime path.

Serve the browser assets from your application. The runtime does not depend on third-party requests.

Your work policy.

Choose the scope, algorithm and work parameters. Match the response to the value and risk of the protected action.

Your acceptance decision.

Verify the proof and lifecycle conditions. Post-solve policy can allow, step up or deny.

From challenge to continuation

Work. Verify. Decide.

01 / APPLICATION

Issue a challenge.

Bind it to a scope and expiry under the application's policy.

02 / BROWSER

Compute the work.

Run the selected solver and produce a response. Show status without a puzzle or human checkbox.

03 / HOST

Check and continue.

Verify the proof, token lifecycle and risk disposition before accepting the action.

Completed work is a computational result. It does not identify a person or guarantee the absence of automation.

The visitor experience

Progress you can read. Recovery you can use.

A static mark, a visible status, one quiet track. Native buttons help when a check fails or expires.

Explore states, languages and widths →

Ready / connecting / checking	Visible text, no invented percentage
Complete / expired / failed	Distinct meaning and recovery
Version mismatch	Reload a coherent asset set
Long labels and RTL	Nine languages plus pt-BR

Privacy belongs in configuration

First-party. Precisely described.

Telemetry is off by default. Strict mode uses no cookies. Standard adaptive behavior is an explicit deployment choice.

Keep the boundary visible.

Configured adaptive modes can use first-party cookies and pseudonymous signals. Retention and rotation depend on signal scope and lifetime.

Read the privacy contract ↗

Higher-value abuse needs a stronger disposition. Default work alone does not price out every valuable attack.

Integration

Choose your stack. Keep one lifecycle.

Core Rust support, Symfony integration and frontend wrappers share the computational check's lifecycle.

Application and server

Challenge issuance, verification, policy and deployment controls.

Server integration →

Frontend

React, Vue, Svelte and web-component wrappers around the shared client.

Frontend integration →

Before you integrate

A few useful answers.

Does KiwiCaptcha require Faktor or Kiwi CI?

No. KiwiCaptcha is a standalone product from the same company.

Does completed work prove the visitor is human?

No. It proves the selected computational work under its rules. Automation can compute work too. The application still verifies the response and decides acceptance.

Can I deploy it without third-party runtime requests?

Yes. The runtime assets are first-party. Choose and document the privacy, telemetry and adaptive policy that fits your deployment.

Where do I start?

Use the repository and integration documentation. The reviewed baseline describes the Rust crate and Symfony bundle as pre-publication; this site links source setup directly.

Explore integration → **View the source ↗**

kiwi captcha

Self-hosted computational checks. First-party control.

Source & documentation ↗

Logo & design assets

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

kiwi captcha

Prior edition / 11 October 2026 Retained design and pinned research. Current source and expanded experience are in the updated edition.

Open current website →

Integration / From source to host

Your application sets the conditions.

Serve the assets, issue the challenge, verify the response and decide whether the action can continue.

01 / ISSUE

Define the scope.

Select the policy, expiry and algorithm. Bound challenge issuance so the check itself stays available.

02 / VERIFY

Keep verification bounded.

Enforce the response's rules, scope and lifetime. Limit concurrent verification work and handle consumption deliberately.

03 / ACCEPT

Apply the disposition.

Post-solve risk policy remains separate from browser completion. Allow, step up or deny according to the action.

Source and documentation

The right entry point for your stack.

Rust core

The core implementation and protocol documentation.

Core source & guide ↗

Symfony

Application integration, privacy configuration and accessibility boundaries.

Symfony source & guide ↗

Frontend wrappers

Shared client lifecycle across React, Vue, Svelte and web components.

Explore frontend packages ↗

Operations

Threat model, cost-to-abuse analysis and deployment limits.

Read security and operation guidance ↗

Links use the reviewed source baseline. Follow its publication and setup notes before choosing an installation path.

The state contract matters.

Pending means deferred work has never started. Idle means a manual restart after reset or credential loss. Running means computation is active. Browser completion means a token was produced; host verification and policy still control acceptance.

Failure and expiry offer native retry. Worker unavailability means the check could not start; execution unavailability means the check could not finish. Version mismatch clears the token and requires a coherent reload.

Explore the widget states →

Match work to the action.

Default work alone does not price out high-value abuse in the documented cost-to-abuse model. Include the higher-risk step-up or deny path and measure the workload you actually deploy.

Document the privacy choice.

First-party delivery and disabled-by-default telemetry are concrete properties. Cookies, pseudonymous signals, retention and rotation remain mode-specific configuration choices.

---

kiwi captcha

Self-hosted computational checks. First-party control.

Source & documentation ↗

Logo & design assets

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI

kiwi captcha

Integration Widget Identity

Prior edition / 11 October 2026 Retained design and pinned research. Current source and expanded experience are in the updated edition.

Open current website →

Widget / Interactive design specimen

A quiet check in every state.

Explore real runtime labels, recovery states, language direction and narrow widths.

Visual simulation only. No challenge, solver, token or host verification runs here.

State

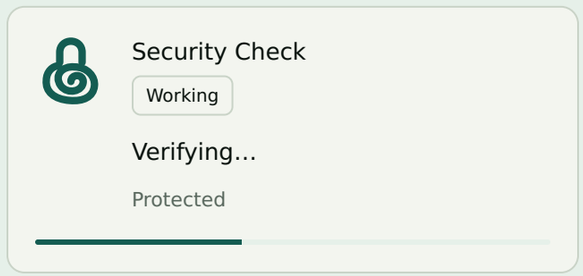
Checking

Language

English

Widget width

320 px



Computational work is active. The track is decorative; no percentage is implied.

☐
200% component text

The small details carry meaning.

The mark stays static. Status text remains visible. One decorative track is hidden from assistive technology. Native recovery buttons retain keyboard access and focus indication.

Completion labels describe local work. They do not certify a person or replace the application's acceptance policy.

Read the integration boundaries →

---

kiwi captcha

Self-hosted computational checks. First-party control.

Source & documentation ↗

Logo & design assets

An independent Bel Consulting product.
Also in the family: Faktor · Kiwi CI